第3回 演習・宿題：学習支援エージェントを作る

この回のゴール

- エージェントが「モデル＋ループ＋ツール」でできていることを、自分のコードで確かめる
- どのツールが、どの引数で呼ばれたかを読める
- ツールを1つ追加できる

準備

前の回までの作業が終わっている場合は、`main` から `lec03` ブランチを作ります。

```
git switch main && git switch -c lec03
```

終わっていない場合は、講座 repo の開始時点のコード（`lec03-start` タグ）からブランチを作れます。

```
git remote -v                      # upstream がなければ次の行で登録する
git remote add upstream https://github.com/matsuolab/lecture-ai-engineering-2026.git
git fetch upstream --tags
git switch -c lec03 lec03-start
```

演習（30分）

1. 第1回と同じ質問をエージェントに投げる

```
uv run python -m app.agent "AIE2026の講座で使う「lecNN-start タグ」とは何ですか？"
```

第1回の `app.ask` の回答と比べてみましょう。

2. ツール呼び出しを読む

```
uv run python -m app.agent --trace "LoRAとQLoRAの違いは？"
```

`[tool call]` の行で、モデルがどのツールをどんな引数で呼んだかを確認します。`[usage]` の `requests` は、1つの質問で LLM を何回呼んだかを表します。

3. ツールを1つ追加する

`app/tools.py` に `make_quiz` を追加し、`app/agent.py` の `TODO(第3回 演習)` の場所で登録します。

- 例：指定した回（`lecture: int`）の用語から1つを選び、「この用語の意味は？」という問題と答えを返す
- docstring には「いつ、何のために使うツールか」を書く（モデルはこれを読んで使うかどうかを決める）

```
uv run python -m app.agent --trace "第2回の内容から確認問題を1問出して"
uv run pytest -q
```

4. 提出

`lectures/03/answer.md` を書き、`lec03` → 自分の fork の `main` へ PR を出します。

宿題（どれか1つで完了）

レベル	内容
基礎	`INSTRUCTIONS` やツールの docstring を書き換え、ツールの呼ばれ方や回答がどう変わるかを `--trace` で比べて記録する
実装	学習の記録を `learning_log/` に保存する `save_learning_log` ツールを作る。保存の前に利用者の確認を挟む（Pydantic AI の `requires_approval=True`）。TestModel を使ったテストも書く
発展	次のどちらか。(a) 同じエージェントを Google の ADK で作り直し、書き方や `adk web` での見え方を比較する。(b) 講義で紹介した Pi を Codespaces 上で動かし、公開されている Pi のシステムプロンプトと自分のエージェントの `INSTRUCTIONS` を比較する

発展（b）の進め方

Pi はファイルの読み書きやコマンドの実行ができるコーディングエージェントで、ツールを実行するたびに確認は求めません。自分のPCではなく、Codespaces の中で試してください。

```
npm install -g --ignore-scripts @earendil-works/pi-coding-agent
pi
```

- Gemini は環境変数 `GEMINI_API_KEY` を読むので、Codespaces secret があればそのまま使えます。Pi の中で `/model` と入力し、Gemini のモデルを選びます
- npm のパッケージ名は `@earendil-works/pi-coding-agent` です（古い記事にある `@mariozechner/...` は旧名です）
- Pi のシステムプロンプトは、公開リポジトリ https://github.com/earendil-works/pi の `packages/coding-agent/src/core/system-prompt.ts` にあります。自分の `INSTRUCTIONS` と比べ、何が書かれていて何が書かれていないかを見てみましょう
- 公式ドキュメント：https://github.com/earendil-works/pi/tree/main/packages/coding-agent/docs

参考

- Pydantic AI: https://ai.pydantic.dev/
- ADK: https://google.github.io/adk-docs/